

웹시스템개발1

실행 컨텍스트



실행 컨텍스트란?

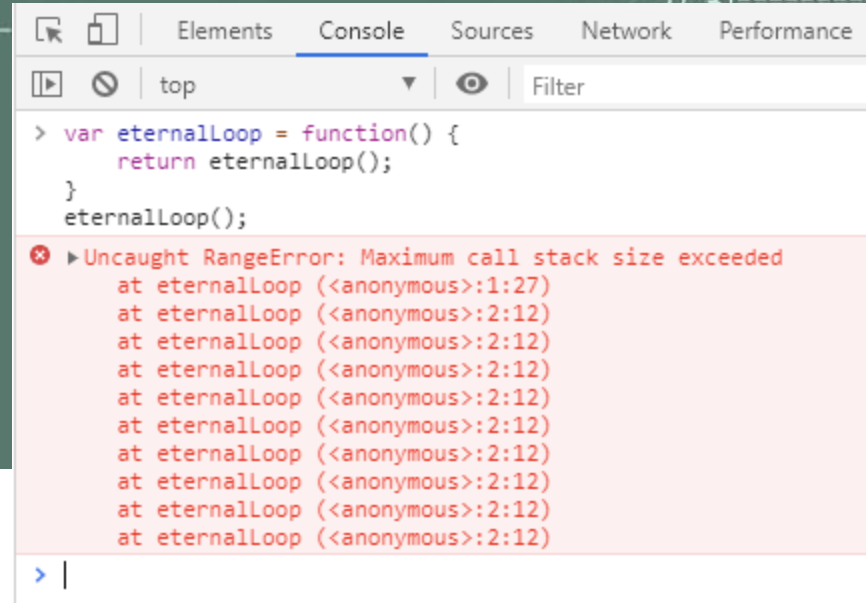
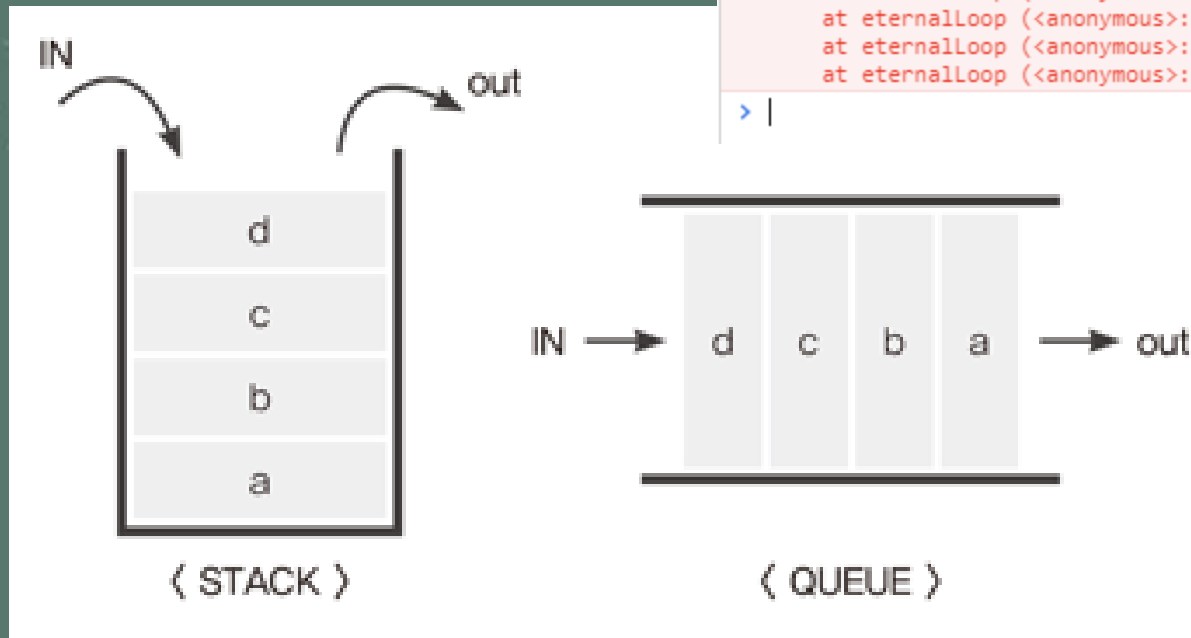


- 실행할 코드에 제공할 환경 정보들을 모아 놓은 객체
- 자바스크립트는 어떤 실행 컨텍스트가 활성화되는 시점에 선언된 변수를 위로 끌어올리고, 외부 환경 정보를 구성하고, `this` 값을 설정하는 등의 동작을 수행

```
function outer(arg) {  
    function inner() {  
    }  
    var v = 'environmentRecord'  
}  
func();
```

실행 컨텍스트란?

● 스택과 큐



실행 컨텍스트란?



- '동일한 환경'에 있는 코드들을 실행할 때 필요한 환경 정보들을 모아 컨텍스트를 구성하고, 이를 호출 스택 (call stack)에 쌓아올렸다가, 가장 위에 쌓여있는 컨텍스트와 관련 있는 코드들을 실행하는 식으로 전체 코드의 환경과 순서를 보장
- '동일한 환경'
 - 하나의 실행 컨텍스트를 구성할 수 있는 방법
 - 전역공간, eval() 함수, 함수
- 흔히 실행 컨텍스트를 구성하는 방법은 함수를 실행하는 것

실행 컨텍스트란?



```
// -----  
var a = 1;  
  
function outer() {  
  function inner() {  
    console.log(a); // undefined  
    var a = 3;  
  }  
  
  inner(); // ----- (2)  
  console.log(a); // 1  
}  
  
outer(); // ----- (3)
```



실행 컨텍스트



● VariableEnvironment

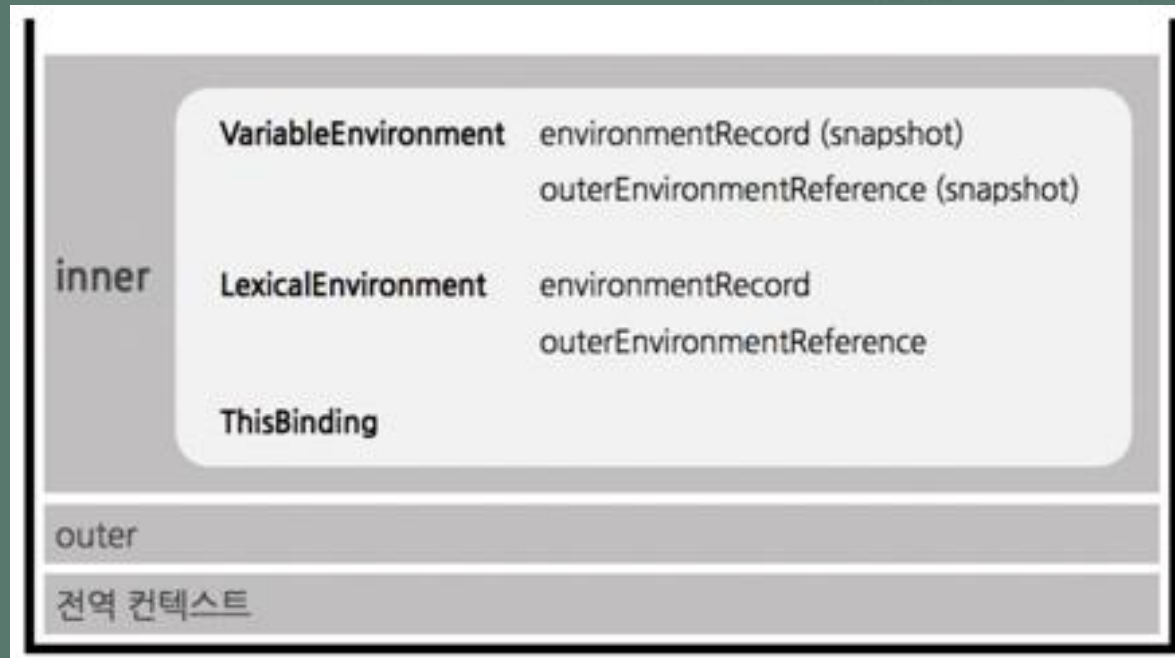
- 현재 컨텍스트 내의 식별자들에 대한 정보 + 외부 환경 정보
- 선언 시점의 LexicalEnvironment의 스냅샷(snapshot)으로, 변경 사항은 반영되지 않음

● LexicalEnvironment

- 처음에는 VariableEnvironment와 같지만 변경 사항이 실시간으로 반영됨

● ThisBinding

- 식별자가 바라봐야 할 대상 객체



LexicalEnvironment



- “컨텍스트의 내부에는 a, b, c 와 같은 식별자들이 있고 그 외부 정보는 D 를 참조하도록 구성돼 있다”라는, 컨텍스트를 구성하는 환경 정보들을 사전에서 접하는 느낌으로 모아 놓은 것
- lexical environment vs LexicalEnvironment
 - 추상적인 개념인 lexical environment를 자바스크립트에서 구현한 구체적 대상이 LexicalEnvironment

environmentRecord와 호이스팅



● environmentRecord에는 현재 컨텍스트와 관련된 코드의 식별자 정보들이 저장됨

- 컨텍스트를 구성하는 함수에 지정된 매개변수 식별자
- 선언한 함수가 있을 경우 그 함수 자체
- var로 선언된 변수의 식별자

```
function outer(arg) {  
    function inner() {  
    }  
    var v = 'environmentRecord'  
}  
func();
```

environmentRecord와 호이스팅



- 컨텍스트 내부 전체를 처음부터 끝까지 쪽 훑어나가며 순서대로 수집함
- 코드가 실행되기 전에 자바스크립트 엔진은 이미 해당 환경에 속한 코드의 변수명을 알고 있게 됨
- 엔진의 실제 동작 방식 대신에 '자바스크립트 엔진은 식별자들을 최상단으로 끌어올려놓은 다음 실제 코드를 실행한다'라고 생각하더라도 문제될 것이 전혀 없음
- 호이스팅 (hoisting)
 - '끌어올리다'라는 의미의 hoist에 ing를 붙여 만든 동명사로, 변수 정보를 수집하는 과정을 이해하기 쉬운 방법으로 대체한 가상의 개념

rd와 호이스팅



```
function a(x) {    // 수집 대상 1

    console.log(x);

    var x;          // 수집 대상 2
    console.log(x);

    var x = 2;      // 수집 대상 3
    console.log(x);
}

a(1);
```

```
function a() {

    var x = 1;      // 수집 대상 1
    console.log(x);

    var x;          // 수집 대상 2
    console.log(x);

    var x = 2;      // 수집 대상 3
    console.log(x);
}

a();
```

```
function a() {

    var x; // 수집 대상 1의 선언 부분
    var x; // 수집 대상 2의 선언 부분
    var x; // 수집 대상 3의 선언 부분

    x = 1; // 수집 대상 1의 할당 부분
    console.log(x);
    console.log(x);

    x = 2; // 수집 대상 3의 할당 부분
    console.log(x);
}

a(1);
```

environmentRecord와 호이스팅



```
function a() {  
    var x; // 수집 대상 1의 선언 부분  
    var x; // 수집 대상 2의 선언 부분  
    var x; // 수집 대상 3의 선언 부분  
  
    x = 1; // 수집 대상 1의 할당 부분  
    console.log(x);  
    console.log(x);  
  
    x = 2; // 수집 대상 3의 할당 부분  
    console.log(x);  
}
```

```
a(1);
```

1. 변수 x 선언
2. 변수 x 선언, 무시
3. 변수 x 선언, 무시
4. x에 1을 할당
5. x를 출력, 1 출력
6. x를 출력, 1 출력
7. x에 2를 할당
8. x를 출력, 2 출력

er

```
function a() {  
  
    console.log(b);  
  
    var b = 'bbb'; // 수집 대상 1  
    console.log(b);  
  
    function b() {} // 수집 대상 2  
  
    console.log(b);  
}  
  
a();
```

```
function a() {  
  
    var b; // 수집 대상 1  
    function b() {} // 수집 대상 2  
  
    console.log(b);  
  
    b = 'bbb';  
    console.log(b);  
    console.log(b);  
}  
  
a();
```

변수와 호이스팅



- 변수는 선언부만 끌어올림
- 함수 선언은 전체를 끌어올림
- 호이스팅이 끝난 상태에서의 함수 선언 문은 함수명으로 선언한 변수에 함수를 할당한 것처럼 여길 수 있음

```
function a() {  
  
    var b;  
    var b = function b() {};  
  
    console.log(b);  
  
    b = 'bbb';  
    console.log(b);  
    console.log(b);  
}  
  
a();
```

environmentRecord와 호이스팅



```
function a() {  
    var b;  
    var b = function b() {};  
  
    console.log(b);  
  
    b = 'bbb'  
    console.log(b);  
    console.log(b);  
}  
  
a();
```

1. 변수 b 선언
2. 변수 b를 선언, 무시
3. 변수 b에 함수 b 할당
4. 변수 b에 할당된 함수 b를 출력
5. b에 'bbb'를 할당
6. b를 출력, 'bbb' 출력
7. b를 출력, 'bbb' 출력

f b() {}

bbb

bbb

environmentRecord와 호이스팅



● 함수 선언문 (function declaration)

- function 정의부만 존재하고 별도의 할당 명령이 없는 것을 의미
- 반드시 함수명이 정의돼 있어야 함

● 함수 표현식

- 정의한 function을 별도의 변수에 할당하는 것을 의미
- 함수명이 없어도 됨

```
function a() {}  
a();  
  
var b = function() {};  
b();  
  
var c = function d() {};  
c();  
d(); // 에러!
```

environmentRecord와 호이스팅



```
console.log(sum(1, 2));  
console.log(multiply(3, 4));
```

```
function sum(a, b) {  
  // 함수 선언문 sum  
  return a + b;  
}
```

```
var multiply = function(a, b) {  
  // 함수 표현식 multiply  
  return a * b;  
};
```

```
var sum = function sum(a, b) {  
  // 함수 선언문은 전체를 호이스팅합니다.  
  return a + b;  
};
```

```
var multiply;  
console.log(sum(1, 2));  
console.log(multiply(3, 4));
```

```
multiply = function(a, b) {  
  // 변수의 할당부는 원래 자리에 남겨둡니다.  
  return a * b;  
};
```

environmentRecord와 호이스팅



```
var sum = function sum(a, b) {  
  // 함수 선언문은 전체를 호이스팅합니다.  
  return a + b;  
};  
  
var multiply;  
console.log(sum(1, 2));  
console.log(multiply(3, 4));  
  
multiply = function(a, b) {  
  // 변수의 할당부는 원래 자리에 남겨둡니다.  
  return a * b;  
};
```

1. 변수 sum 선언
2. 변수 multiply 선언
3. 변수 sum에 함수 sum 할당
4. sum 실행, 3 출력
5. 에러 메시지 출력

3

✖ ▶ Uncaught TypeError: multiply is not a function
at <anonymous>:8:13

environmentRecord와 호이스팅



● 함수 선언문의 위험성

```
...
60 console.log(sum(3, 4));    // Uncaught Type Error: sum is not a function
...

100 var sum = function (x, y) {
101     return x + y;
102 };
...

200 var a = sum(1, 2);
...

5000 var sum = function (x, y) {
5001     return x + ' + ' + y + ' = ' + (x + y);
5002 };
...

5010 var c = sum(1, 2);
5011 console.log(c);
...
```

```
...
60 console.log(sum(3, 4));
...

100 function sum (x, y) {
101     return x + y;
102 }
...

200 var a = sum(1, 2);
...

5000 function sum (x, y) {
5001     return x + ' + ' + y + ' = ' + (x + y);
5002 }
...

5010 var c = sum(1, 2);
5011 console.log(c);
...
```

스코프, 스코프 체인 outerEnvironmentReference



● 스코프

- 식별자에 대한 유효범위
- 어떤 경계 A의 외부에서 선언한 변수는 A의 내부에서도 접근이 가능하지만, A의 내부에서 선언한 변수는 오직 A의 내부에서만 접근할 수 있음

● ES5까지의 자바스크립트는 특이하게도 전역공간을 제외하면 오직 함수에 의해서만 스코프가 생성

● ES6에서는 블록에 의해서도 스코프가 생성

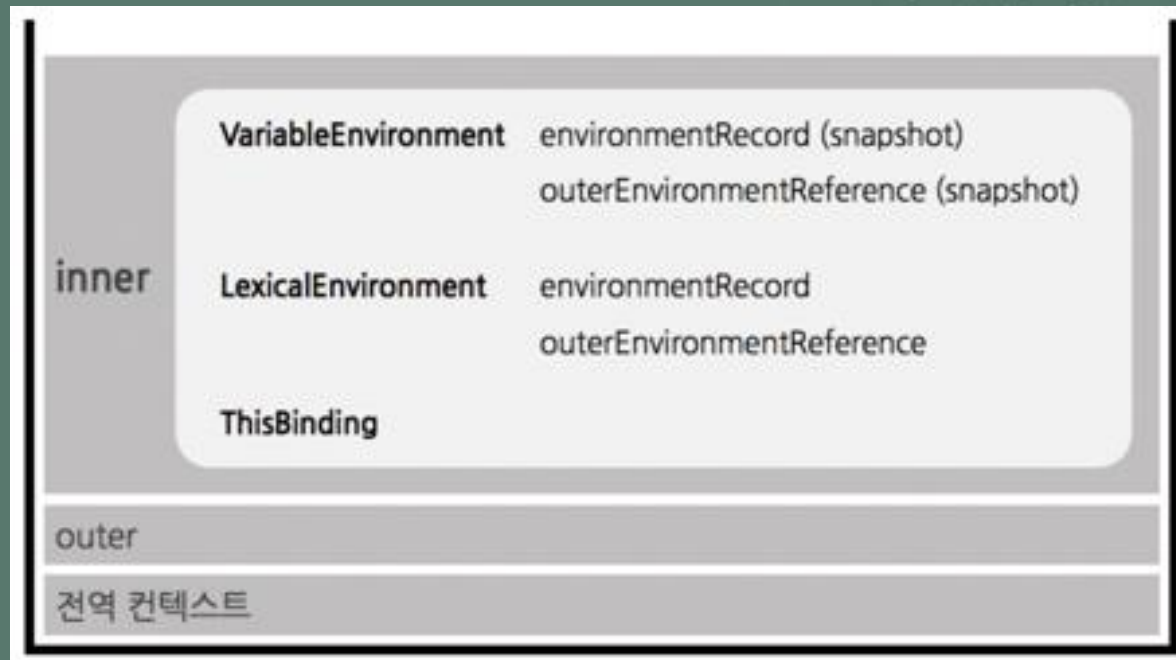
- var로 선언한 변수에 대해서는 작용하지 않음
- let, const, class, strict mode에서의 함수 선언 등에 대해서만 작용
- ES6에서는 둘을 구분하기 위해 함수 스코프, 블록 스코프라는 용어를 사용

스코프, 스코프 체인 outerEnvironmentReference



● 스코프 체인 (scope chain)

- '식별자의 유효범위'를 안에서부터 바깥으로 차례로 검색해나가는 것
- 검색을 가능하게 하는 것이 outerEnvironmentReference



스코프, 스코프 체인

outerEnvironmentReference



● 스코프 체인 (scope chain)

- outerEnvironmentReference는 현재 호출된 함수가 '선언될 당시'의 LexicalEnvironment를 참조
- outerEnvironmentReference는 연결리스트 (linked list) 형태를 가짐

```
var a = 1;

var outer = function() {
  var inner = function() {
    console.log(a);
    var a = 3;
  };

  inner();
  console.log(a);
};

outer();
console.log(a);
```

1. 전역 컨텍스트 활성화
 - 전역 컨텍스트의 environmentRecord에 { a, outer } 식별자 저장
 - 전역 컨텍스트는 선언 시점이 없으므로, 전역 컨텍스트의 outerEnvironmentReference에는 아무것도 담기지 않음
2. 전역 스코프에 있는 변수 a에 1을, outer에 함수를 할당
3. outer 함수 호출, outer 실행 컨텍스트 활성화
 - outer 실행 컨텍스트의 environmentRecord에 { inner } 식별자 저장
 - outerEnvironmentReference에는 outer 함수가 선언될 당시의 (outer 함수는 전역 공간에서 선언됐으므로) 전역 컨텍스트의 LexicalEnvironment를 참조 복사

스코프, 스코프 체인

outerEnvironmentReference



```
var a = 1;

var outer = function() {
  var inner = function() {
    console.log(a);
    var a = 3;
  };

  inner();
  console.log(a);
};

outer();
console.log(a);
```

4. outer 스코프에 있는 변수 inner에 함수를 할당
5. inner 함수 호출, inner 실행 컨텍스트 활성화
 - inner 실행 컨텍스트의 environmentRecord에 { a } 식별자 저장
 - outerEnvironmentReference에는 inner 함수가 선언될 당시의 (inner 함수는 outer 함수 내부에서 선언됐으므로) outer 컨텍스트의 LexicalEnvironment를 참조 복사
6. 식별자 a에 접근
 - 현재 활성화 상태인 inner 컨텍스트의 environmentRecord에서 a를 검색
 - a가 발견되었지만 아직 할당된 값이 없으므로 undefined 출력
7. inner 스코프에 있는 변수 a에 3을 할당
8. inner 함수 실행 종료, inner 실행 컨텍스트가 호출 스택에서 제거, outer 실행 컨텍스트 활성화

스코프, 스코프 체인 outerEnvironmentReference



```
var a = 1;

var outer = function() {
  var inner = function() {
    console.log(a);
    var a = 3;
  };

  inner();
  console.log(a);
};

outer();
console.log(a);
```

9. 식별자 a에 접근
 - 현재 활성화 상태인 outer 컨텍스트의 environmentRecord에서 a를 검색했으나 없으므로, outer 컨텍스트의 outerEnvironmentReference에 있는 (전역 컨텍스트의) environmentRecord에서 a를 검색
 - a에 저장된 값 1 출력
7. outer 함수 실행 종료, outer 실행 컨텍스트가 호출 스택에서 제거, 전역 컨텍스트 활성화
8. 식별자 a에 접근
 - 현재 활성화 상태인 전역 컨텍스트의 environmentRecord에서 a를 검색
 - a에 저장된 값 1 출력
9. 모든 코드 실행 완료, 전역 컨텍스트가 호출 스택에서 제거

L.E: Lexical Environment
 e: environmentRecord
 o: outerEnvironmentReference

```
var a = 1;

var outer = function() {
  var inner = function() {
    console.log(a);
    var a = 3;
  };

  inner();
  console.log(a);
};

outer();
console.log(a);
```

[0] 전역 컨텍스트 활성화 - LexicalEnvironment, VariableEnvironment, thisBinding

[1], [2] a에 1, outer에 함수 할당

[10] outer 함수 호출. 전역 컨텍스트 비활성화

[2] outer 실행 컨텍스트 활성화

[3] inner에 함수 할당

[7] inner 함수 호출. outer 실행 컨텍스트 비활성화

[3] inner 실행 컨텍스트 활성화

전역 컨텍스트

L.E	
e	a, outer

(this: 전역 객체)

outer 컨텍스트

L.E	
e	inner
o	GLOBAL L.E

(this: 전역 객체)

inner 컨텍스트

L.E	
e	a
o	outer L.E

(this: 전역 객체)

[4] inner의 L.E에서 a 탐색
→ undefined 출력

[5] a에 3 할당

[6] inner 함수 종료. inner 실행 컨텍스트 제거

[7] outer 실행 컨텍스트 재활성화

[8] outer의 L.E에서 a 탐색
→ GLOBAL의 L.E에서 a 탐색 → 1 출력

[9] outer 함수 종료. outer 실행 컨텍스트 제거

[10] 전역 컨텍스트 재활성화

[11] GLOBAL의 L.E에서 a 탐색 → 1 출력

[] 전체 종료. 전역 컨텍스트 제거

스코프, 스코프 체인 outerEnvironmentReference



```
var a = 1;

var outer = function() {
  var inner = function() {
    console.log(a);
    var a = 3;
  };

  inner();
  console.log(a);
};

outer();
console.log(a);
```

● 스코프 체인을 타고 접근 가능한 변수가 늘어남

- 전역 공간
 - 전역 스코프에서 생성된 변수에만 접근 가능
- outer 함수 내부
 - outer 및 전역 스코프에서 생성된 변수에 접근 가능
 - inner 스코프에서 생성된 변수 접근 불가
- inner 함수 내부
 - inner, outer, 전역 스코프 모두 접근 가능

스코프, 스코프 체인 outerEnvironmentReference



```
var a = 1;

var outer = function() {
  var inner = function() {
    console.log(a);
    var a = 3;
  };

  inner();
  console.log(a);
};

outer();
console.log(a);
```

● 변수 은닉화 (variable shadowing)

- inner 함수 내부에서 a에 접근하려고 하면?
 - 무조건 스코프 체인 상의 첫 번째 인자, 즉 inner 스코프의 LexicalEnvironment에 a 식별자가 존재하므로 스코프 체인 검색을 더 진행하지 않고, 즉시 inner LexicalEnvironment에 a를 반환
 - 전역 공간에서 선언한 동일한 이름의 a 변수에는 접근 불가

스코프, 스코프 체인 outerEnvironmentReference



● 스코프 체인 확인 - 크롬 전용

- 스코프 체인 중 현재 실행 컨텍스트를 제외한 상위 스코프 정보들 확인
- 함수 내부에서 실제로 호출되는 외부 변수들의 정보만 보여줌

```
var outer = function() {  
  
    var a = 1;  
    var b = 2;  
    var c = 3  
  
    var inner = function() {  
        console.log(b);  
        console.dir(inner);  
    };  
  
    inner();  
};  
  
outer();
```

```
▼ f inner() ⓘ  
  length: 0  
  name: "inner"  
  arguments: null  
  caller: null  
  ▶ prototype: {constructor: f}  
  ▶ __proto__: f ()  
    [[FunctionLocation]]: VM467:7  
  ▼ [[Scopes]]: Scopes[2]  
    ▼ 0: Closure (outer)  
      b: 2  
      ▶ inner: f ()  
    ▶ 1: Global {parent: Window, opener: null, top: Window, length: 0, frames: Window, ...}
```

스코프, 스코프 체인 outerEnvironmentReference



● 스코프 체인 확인 - debugger

```
var outer = function() {  
  
    var a = 1;  
    var b = 2;  
    var c = 3  
  
    var inner = function() {  
  
        console.log(b);  
        debugger;  
    };  
  
    inner();  
};  
  
outer();
```

Debugger paused

▶ Watch

▼ Call Stack

inner	VM478:10
outer	VM478:13
(anonymous)	VM478:16

▼ Scope

▼ Local

▶ this: Window

▼ Closure (outer)

b: 2

▶ Global

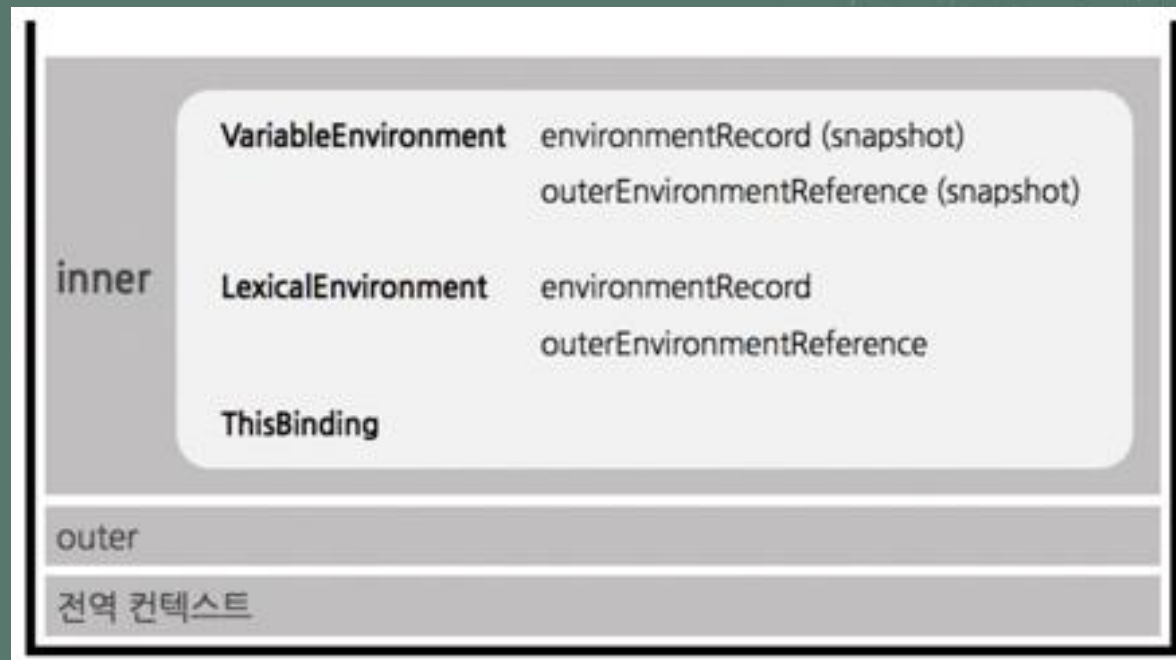
Window

```
1 var outer = function() {  
2  
3     var a = 1;  
4     var b = 2;  
5     var c = 3  
6  
7     var inner = function() {  
8  
9         console.log(b);  
10        debugger;  
11    };  
12  
13    inner();  
14 };  
15  
16 outer();
```

this



- 실행 컨텍스트의 ThisBinding에는 this로 지정된 객체가 저장됨
- 실행 컨텍스트 활성화 당시에 this가 지정되지 않은 경우 this에는 전역 객체가 저장됨



정리



- 실행 컨텍스트는 실행할 코드에 제공할 환경 정보들을 모아놓은 객체
- 실행 컨텍스트 객체는 활성화되는 시점에 `VariableEnvironment`, `LexicalEnvironment`, `ThisBinding` 세 가지 정보를 수집
- `environmentRecord`
 - 매개변수명, 변수의 식별자, 선언한 함수의 함수명 등을 수집하는
- `outerEnvironmentReference`
 - 직전 컨텍스트의 `LexicalEnvironment` 정보를 참조
- 호이스팅
 - `environmentRecord`의 수집 과정을 추상화한 개념으로, 실행 컨텍스트가 관여하는 코드 집단의 최상단으로 '끌어올린다'고 해석하는 것
- 스코프
 - 변수의 유효범위
 - `outerEnvironmentReference`는 해당 함수가 선언된 위치의 `LexicalEnvironment`를 참조



Thank you!

